

STUDENT PLACEMENT PREDICTION USING MACHINE LEARNING

1. Abstract

Student placement is an important outcome for educational institutions and students. Placement depends on several academic, technical, professional, and extracurricular factors. This project develops a machine learning-based system for predicting whether a student is likely to be placed.

The project uses a dataset containing 8,000 student records and 28 original attributes, including CGPA, attendance, coding performance, aptitude, communication skills, technical skills, projects, internships, certifications, hackathons, GitHub projects, resume score, soft skills, leadership, training hours, mock interview performance, and preferred career domain.

Three classification algorithms—Logistic Regression, Decision Tree, and Random Forest—were implemented and compared. The dataset was divided into training and testing sets using an 80:20 split. Numerical features were standardized and categorical features were encoded using a preprocessing pipeline.

The dataset has an overall placement rate of 20.38%. Among the evaluated models, Logistic Regression and Random Forest achieved an accuracy of approximately 79.63%, while the Decision Tree achieved approximately 79.00%. The project also provides placement probability categories and personalized recommendations for improving a student's placement profile.

2. Introduction

Campus placement is one of the major concerns for students pursuing higher education. A student's placement outcome can depend on many factors rather than academic performance alone. Factors such as coding ability, aptitude, communication, technical knowledge, internships, projects, certifications, interview performance, and extracurricular achievements can influence employability.

Traditional placement analysis generally involves manually examining student profiles. With a large number of students, this process can become time-consuming and difficult to perform consistently.

Machine learning provides an opportunity to analyze historical student data and identify patterns associated with placement outcomes. A classification model can learn from existing student records and predict whether a new student is likely to be placed.

This project therefore focuses on developing a machine learning-based Student Placement Prediction System. The system analyzes multiple student characteristics and produces a placement prediction along with a probability-based category.

3. Problem Statement

The objective of this project is to develop a machine learning system capable of predicting student placement outcomes using academic, technical, professional, and extracurricular attributes.

The system should:

- Analyze historical student placement data.
 - Identify important factors associated with placement.
 - Preprocess numerical and categorical student information.
 - Train multiple machine learning classification models.
 - Compare the performance of the models.
 - Predict whether a student is likely to be placed.
 - Estimate placement probability.
 - Categorize students into High, Medium, or Low placement probability.
 - Provide recommendations for improving weak areas of a student's profile.
-

4. Objectives

The major objectives of the project are:

1. To analyze student placement data using Python and machine learning.
2. To identify the important factors related to student placement.
3. To perform exploratory data analysis on the dataset.
4. To preprocess numerical and categorical variables.
5. To develop classification models for placement prediction.
6. To compare Logistic Regression, Decision Tree, and Random Forest models.
7. To evaluate the models using accuracy, precision, recall, and F1-score.
8. To calculate placement probabilities for students.

9. To classify students according to their placement probability.
 10. To generate personalized recommendations for improving placement readiness.
-

5. Scope of the Project

The project focuses on predicting student placement outcomes from student-related attributes.

The system can be useful for:

- Students preparing for campus placements.
- Educational institutions.
- Training and placement departments.
- Career guidance systems.
- Academic mentors.

Students can use the system to understand areas where improvement may increase their placement readiness. Institutions can potentially use similar systems to identify students who may require additional training.

The current project is based on the available dataset and should therefore be considered a predictive analytical system rather than a guaranteed placement decision-making system.

6. Technologies Used

The following technologies and libraries were used:

Programming Language

- Python

Data Processing

- Pandas
- NumPy

Data Visualization

- Matplotlib

- Seaborn

Machine Learning

- Scikit-learn

Machine Learning Algorithms

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier

Evaluation Metrics

- Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
 - Classification Report
 - ROC Curve / ROC-AUC analysis
-

7. Dataset Description

The project uses the dataset:

`student_placement_train.csv`

The dataset contains:

- **8,000 student records**
- **28 original columns**

The target variable is:

- `placed`

where:

- `0` = Not Placed
- `1` = Placed

The dataset contains both numerical and categorical attributes.

Major Dataset Attributes

Attribute	Description
student_id	Unique identifier of the student
age	Age of the student
gender	Gender of the student
cgpa	Academic CGPA
attendance_percentage	Attendance percentage
backlogs	Number of backlogs
coding_score	Coding performance score
aptitude_score	Aptitude performance score
communication_score	Communication performance score
technical_score	Technical skill score
projects_count	Number of projects
major_projects	Number of major projects
internships_count	Number of internships
internship_months	Internship duration
certifications_count	Number of certifications
hackathons_participated	Number of hackathons participated in
hackathons_won	Number of hackathons won
coding_platform_score	Coding platform performance
github_projects	Number of GitHub projects
linkedin_score	LinkedIn profile score
resume_score	Resume score

soft_skills_score	Soft-skills score
leadership_score	Leadership score
extracurricular_score	Extracurricular activity score
training_hours	Training hours
mock_interview_score	Mock interview score
preferred_domain	Preferred career domain
placed	Placement outcome

8. Dataset Statistics

The dataset contains 8,000 observations.

The target distribution is:

- Not Placed: **79.625%**
- Placed: **20.375%**

Therefore, the overall placement rate in the dataset is approximately:

20.38%

The dataset is therefore imbalanced, with significantly more students belonging to the non-placed class than the placed class.

Some important dataset averages are:

Feature	Average
CGPA	7.58
Coding Score	53.16
Internship Count	0.92
Projects	2.60
Placement Rate	20.38%

9. Data Preprocessing

Data preprocessing is an important stage before applying machine learning algorithms.

The following steps were performed.

9.1 Checking Dataset Dimensions

The dataset was inspected to determine its size.

- Rows: 8,000
- Original columns: 28

After feature engineering, the model input contained 29 features after removing the target and student ID.

9.2 Checking Data Types

The dataset contains:

- Numerical variables such as CGPA, coding score, aptitude score, projects, internships, etc.
- Categorical variables such as gender and preferred domain.

The categorical features identified were:

- `gender`
- `preferred_domain`

The remaining model features were numerical.

9.3 Missing Value Analysis

Missing values were identified in several columns.

The recorded missing values included:

Feature	Missing Values
attendance_percentage	206
coding_score	141
aptitude_score	119
communication_score	149
technical_score	131
internship_months	238
linkedin_score	315
resume_score	189
soft_skills_score	154
training_hours	241
mock_interview_score	295

A missing-value visualization was also generated using a heatmap.

Note: The notebook imports `SimpleImputer`, but the final displayed preprocessing pipeline does not explicitly include the imputer. Therefore, the report does not claim that missing values were imputed.

9.4 Duplicate Checking

Duplicate records were checked using Pandas.

The result was:

Duplicate rows: 0

Therefore, no duplicate rows were identified in the dataset.

9.5 Removal of Student ID

The `student_id` column was removed before model training because it is an identifier rather than a meaningful predictive feature.

The target column `placed` was separated from the input features.

X = Features

y = placed

10. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the dataset and investigate relationships between student characteristics and placement.

Several visualizations were generated.

10.1 Placement Distribution

A count plot was used to visualize the number of placed and non-placed students.

The analysis showed that the majority of students were not placed.

The dataset placement rate was:

20.38%

10.2 Gender and Placement

A cross-tabulation was created to analyze placement percentages according to gender.

This allowed the placement distribution of different gender categories to be examined.

10.3 CGPA vs Placement

A box plot was created to compare CGPA distributions between placed and non-placed students.

This analysis helps examine whether academic performance differs between the two placement categories.

10.4 Coding Score vs Placement

Coding scores were compared between placed and non-placed students using a box plot.

Coding ability is an important part of technical placement preparation and was therefore included in the exploratory analysis.

10.5 Aptitude Score vs Placement

A box plot was used to compare aptitude scores for placed and non-placed students.

This helps examine the relationship between aptitude performance and placement outcomes.

10.6 Internships vs Placement

The number of internships was compared with placement outcomes using a count plot.

This provides insight into whether students with internship experience show different placement patterns.

10.7 Projects vs Placement

The number of projects was analyzed against placement outcomes.

Projects represent practical exposure and were included as one of the features used by the machine learning models.

10.8 Training Hours vs Placement

Training hours were compared between the two placement categories.

The purpose was to understand whether training exposure was associated with placement outcomes.

10.9 Mock Interview Score vs Placement

Mock interview scores were visualized against placement outcomes.

This provides an indication of the relationship between interview preparation and placement.

11. Placement by Preferred Domain

Placement rates were calculated for different preferred domains.

Preferred Domain	Not Placed (%)	Placed (%)
AI/ML	78.84	21.16
Business Analytics	78.36	21.64
Cloud Computing	78.33	21.67
Cyber Security	82.07	17.93
Data Science	81.12	18.88
Software Development	79.24	20.76

The highest placement percentage in the dataset was observed for **Cloud Computing at approximately 21.67%**, followed closely by Business Analytics and AI/ML.

The lowest placement percentage was observed for **Cyber Security at approximately 17.93%**.

These values describe the dataset and should not be interpreted as general industry placement rates.

12. Correlation Analysis

A correlation matrix was generated for the numerical variables.

A heatmap was used to visualize the correlations.

The correlation of individual numerical features with the target variable `placed` was also calculated.

This analysis helps identify variables that show stronger or weaker statistical relationships with the placement outcome.

Correlation, however, does not necessarily imply causation.

13. Feature Engineering

Three additional features were created to improve the representation of student profiles.

13.1 Total Experience

The following feature was created:

```
total_experience =  
internship_months + training_hours / 10
```

This combines internship duration and training exposure into a single experience-related measure.

13.2 Achievement Score

The achievement score was calculated using:

```
achievement_score =  
certifications_count  
+ hackathons_won  
+ major_projects  
+ github_projects
```

This combines several achievement-related indicators into one feature.

13.3 Overall Skill Score

The overall skill score was calculated as:

```
overall_skill_score =  
(coding_score  
+ aptitude_score  
+ communication_score  
+ technical_score  
+ soft_skills_score) / 5
```

This provides an aggregated representation of several important student skills.

14. Feature Selection

The target variable `placed` and identifier `student_id` were removed from the input features.

The resulting dataset contained:

- **8,000 observations**
- **29 model features**

The features consisted of:

Numerical Features

- age
- cgpa
- attendance_percentage
- backlogs
- coding_score
- aptitude_score
- communication_score
- technical_score

- projects_count
- major_projects
- internships_count
- internship_months
- certifications_count
- hackathons_participated
- hackathons_won
- coding_platform_score
- github_projects
- linkedin_score
- resume_score
- soft_skills_score
- leadership_score
- extracurricular_score
- training_hours
- mock_interview_score
- total_experience
- achievement_score
- overall_skill_score

Categorical Features

- gender
 - preferred_domain
-

15. Train-Test Split

The dataset was divided into training and testing sets.

The following configuration was used:

Training Data: 80%

Testing Data: 20%

The resulting dimensions were:

- Training data: **6,400 records**
- Testing data: **1,600 records**

Stratified splitting was used so that the target class distribution was maintained between training and testing data.

16. Feature Preprocessing

A `ColumnTransformer` was used to apply different preprocessing methods to numerical and categorical variables.

Numerical Features

Numerical features were standardized using:

StandardScaler

Standardization helps bring numerical features onto a comparable scale.

Categorical Features

Categorical features were converted into numerical representations using:

OneHotEncoder

The encoder was configured with:

```
handle_unknown = "ignore"
```

This allows the model to handle previously unseen categorical values during prediction.

17. Machine Learning Methodology

Three classification algorithms were implemented.

The models were:

1. Logistic Regression
2. Decision Tree Classifier
3. Random Forest Classifier

Each model was implemented using a Scikit-learn Pipeline containing the preprocessing stage and classifier.

This ensures that the same preprocessing procedure is applied when training and predicting.

18. Logistic Regression

Logistic Regression was used as one of the baseline classification algorithms.

The model was configured with:

```
max_iter = 1000
```

The model was trained on the training dataset and evaluated using the testing dataset.

Results

Metric	Score
Accuracy	79.63%
Precision	0.00
Recall	0.00
F1 Score	0.00

The classification report showed:

- Class 0 had recall of 1.00.
- Class 1 had precision, recall, and F1-score of 0.00.

This indicates that the high accuracy is largely influenced by the majority non-placement class.

19. Decision Tree

A Decision Tree Classifier was implemented with:

```
max_depth = 6  
random_state = 42
```

The model creates decision rules based on feature values to classify students.

Results

Metric	Score
Accuracy	79.00%
Precision	14.29%
Recall	0.61%
F1 Score	1.18%

Although the accuracy was relatively high, the recall for the placed class was very low.

20. Random Forest

Random Forest was implemented using an ensemble of decision trees.

The configuration used was:

```
n_estimators = 200
max_depth = 10
random_state = 42
n_jobs = -1
```

The Random Forest model was trained using the same preprocessing pipeline.

Results

Metric	Score
Accuracy	79.63%
Precision	0.00
Recall	0.00
F1 Score	0.00

The model also produced placement probabilities using `predict_proba()`.

21. Model Comparison

The three models produced the following results:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	79.63%	0.00	0.00	0.00
Decision Tree	79.00%	14.29%	0.61%	1.18%
Random Forest	79.63%	0.00	0.00	0.00

A bar chart was generated in the notebook to visually compare the models.

22. Model Evaluation

Multiple evaluation techniques were used.

22.1 Accuracy

Accuracy represents the proportion of total predictions that were correct.

The models achieved approximately 79% accuracy.

However, accuracy alone is not sufficient for this dataset because the classes are imbalanced.

22.2 Precision

Precision measures how many students predicted as placed were actually placed.

The Decision Tree achieved a precision of 0.1429, while Logistic Regression and Random Forest recorded 0.0.

22.3 Recall

Recall measures how many of the actually placed students were correctly identified.

The Decision Tree achieved a recall of only 0.0061.

Logistic Regression and Random Forest achieved 0.0 recall for the placed class.

22.4 F1 Score

F1-score combines precision and recall.

The Decision Tree achieved:

F1 Score = 0.0118

Logistic Regression and Random Forest achieved:

F1 Score = 0.0

22.5 Confusion Matrix

Confusion matrices were generated for Logistic Regression and Random Forest.

The confusion matrices provide information about:

- True Negatives
- False Positives
- False Negatives
- True Positives

They are particularly useful for understanding the poor detection of the minority placed class.

23. Important Observation Regarding Model Performance

One of the most important findings from the project is that the approximately 79–80% accuracy should **not** be interpreted as an 80% reliable placement prediction system.

The dataset contains approximately:

79.63% non-placed students

Therefore, a model that predominantly predicts the majority class can obtain approximately 79% accuracy without successfully identifying placed students.

This is clearly reflected in the model results:

- Logistic Regression recall for placed students = **0**
- Random Forest recall for placed students = **0**
- Decision Tree recall for placed students = **0.61%**

Therefore, future versions of the project should focus more strongly on minority-class performance rather than accuracy alone.

24. Feature Importance

Feature importance was calculated for the Random Forest model.

The model's feature importance values were extracted using:

```
random_forest.named_steps["classifier"].feature_importances_
```

The transformed feature names were obtained from the preprocessing pipeline.

The top 15 features were then selected and visualized using a bar chart titled:

"Top Factors Influencing Student Placement"

This analysis provides an indication of which features the Random Forest model considered more useful during prediction.

25. Placement Probability Prediction

The Random Forest model was also used to calculate the probability that a student would be placed.

The probability was obtained using:

```
random_forest.predict_proba(X_test)[:, 1]
```

The resulting information includes:

- Actual placement status
- Predicted placement status
- Placement probability

This allows the system to provide more information than a simple binary prediction.

26. Placement Probability Categories

The system categorizes students according to their predicted placement probability.

High Placement Probability

Probability $\geq 75\%$

Medium Placement Probability

$50\% \leq \text{Probability} < 75\%$

Low Placement Probability

Probability $< 50\%$

This categorization provides an intuitive interpretation of the model's probability output.

27. Personalized Recommendations

The project contains a recommendation system that identifies areas where a student may need improvement.

The recommendation rules include:

Coding

If:

`coding_score < 60`

Recommendation:

Improve coding and problem-solving skills.

Aptitude

If:

`aptitude_score < 60`

Recommendation:

Practice quantitative and logical aptitude.

Communication

If:

`communication_score < 60`

Recommendation:

Improve communication and interview skills.

Technical Skills

If:

`technical_score < 60`

Recommendation:

Strengthen core technical concepts.

Projects

If:

`projects_count < 2`

Recommendation:

Build more practical projects.

Internship

If:

`internships_count == 0`

Recommendation:

Try to gain internship experience.

Certifications

If:

`certifications_count == 0`

Recommendation:

Consider completing relevant certifications.

Mock Interviews

If:

`mock_interview_score < 60`

Recommendation:

Practice mock interviews.

If none of these conditions are satisfied, the system recommends continuing interview preparation.

28. Sample Prediction

The notebook demonstrates prediction using a sample student from the test dataset.

The sample output was:

Placement Prediction: 0

Placement Probability: 20.16%

Category: Low Placement Probability

According to the system's classification rules, the sample student was therefore categorized as having a **Low Placement Probability**.

29. Key Project Insights

The notebook reports the following major dataset-level insights:

- Overall Placement Rate: **20.38%**
- Average CGPA: **7.58**
- Average Coding Score: **53.16**
- Average Internship Count: **0.92**
- Average Projects: **2.60**

These values provide a general overview of the student population represented in the dataset.

30. Advantages of the Proposed System

The proposed system has several advantages:

1. It automates placement prediction.
 2. It analyzes multiple student attributes simultaneously.
 3. It supports both numerical and categorical features.
 4. It compares multiple machine learning algorithms.
 5. It provides probability-based predictions.
 6. It categorizes students according to placement probability.
 7. It provides personalized improvement recommendations.
 8. It can help students identify areas requiring additional preparation.
 9. It can support data-driven placement analysis.
-

31. Limitations

The current implementation has several limitations.

31.1 Class Imbalance

Only approximately 20.38% of the students are placed, while approximately 79.63% are not placed.

This significantly affects model performance.

31.2 Low Minority-Class Recall

The models show extremely low recall for the placed class.

Therefore, the system cannot currently be considered a highly reliable placement predictor.

31.3 Accuracy Can Be Misleading

The approximately 79% accuracy is largely influenced by the majority class.

Precision, recall, and F1-score provide a more meaningful evaluation for this problem.

31.4 Dataset Dependence

The predictions depend heavily on the quality and representativeness of the dataset.

31.5 Placement Is Influenced by External Factors

Actual placement can depend on factors not represented in the dataset, such as:

- Company requirements

- Number of vacancies
- Interview difficulty
- Economic conditions
- Candidate preferences
- Hiring policies

Therefore, model predictions should not be considered guarantees.

32. Future Scope

The project can be improved in several ways.

32.1 Handle Class Imbalance

Techniques such as:

- SMOTE
- Random oversampling
- Random undersampling
- Class weights

could be investigated to improve identification of the placed class.

32.2 Hyperparameter Optimization

Algorithms can be optimized using:

- Grid Search
- Randomized Search
- Cross-validation

This may improve predictive performance.

32.3 Additional Algorithms

Future versions could compare additional algorithms such as:

- Support Vector Machine
 - K-Nearest Neighbors
 - Gradient Boosting
 - XGBoost
 - LightGBM
-

32.4 Improved Evaluation

Because of class imbalance, future evaluation should emphasize:

- Precision
- Recall
- F1-score
- ROC-AUC
- Precision-Recall AUC
- Confusion Matrix

rather than relying mainly on accuracy.

32.5 Real-Time Web Application

The model could be integrated into a web application where students enter their profile information and receive:

- Placement probability
 - Placement category
 - Strength analysis
 - Weakness analysis
 - Personalized recommendations
-

32.6 Explainable AI

Explainable AI techniques could be incorporated to explain why a student received a particular prediction.

Methods such as SHAP or LIME could be considered.

33. Conclusion

This project presents a machine learning-based approach for analyzing and predicting student placement outcomes.

A dataset containing 8,000 student records was analyzed using exploratory data analysis and machine learning techniques. Three classification algorithms—Logistic Regression, Decision Tree, and Random Forest—were developed and evaluated.

The dataset showed an overall placement rate of 20.38%. The models achieved approximately 79% accuracy; however, detailed evaluation showed that the models had very poor recall for the placed class. This demonstrates the importance of considering class imbalance and using metrics beyond accuracy.

The project also extends beyond simple prediction by calculating placement probabilities, categorizing students into high, medium, and low probability groups, and generating recommendations based on individual student characteristics.

Although the current model requires further improvement before being used as a dependable placement prediction system, the project demonstrates how machine learning can be applied to student placement analysis and career-readiness assessment.

34. References

1. Scikit-learn Documentation – Machine Learning Algorithms and Evaluation Metrics.
 2. Pandas Documentation – Data Manipulation and Analysis.
 3. NumPy Documentation – Numerical Computing in Python.
 4. Matplotlib Documentation – Data Visualization.
 5. Seaborn Documentation – Statistical Data Visualization.
 6. Python Documentation – Python Programming Language.
-

35. Appendix – Main Libraries Used

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    RocCurveDisplay
)
```

36. Appendix – Machine Learning Workflow

The overall workflow of the project can be summarized as:

Dataset Collection

↓

Data Loading

↓

Data Inspection

↓

Missing Value Analysis

↓

Duplicate Checking



Exploratory Data Analysis



Feature Engineering



Feature Selection



Train-Test Split



Numerical Scaling + Categorical Encoding



Model Training



Logistic Regression / Decision Tree / Random Forest



Model Evaluation



Placement Probability Prediction



Placement Category



Personalized Recommendations